Quality Engineering Report: Test Cases Definition

ADRIAN RODENAS & ADRIAN CASTELAR

1. Introduction

This document presents two complete sets of functional test cases for the two core conversion functions of the Roman Numeral Converter web application developed in Task 1: intToRoman(num) and romanToInt(str). Both functions are pure Java scrips functions that handle the mathematical conversion logic.

The valid domain for Roman numerals is strictly 1-3999(integers only). There is no Roman representation for 0, negative numbers, frations, or values above 3999.

Functions under test:

- IntToRoman(num): Converts a positive integer to its Roman numeral String.
- RomanToInt(str): Converts a valid Roman numeral string to its integre value.

2. Manually Designed Test Cases

Metodologhy:

To design this first set of test cases, I utilized two primary black-box testing techniques: **Domain Partitioning (Equivalence Partitioning)** and **Boundary Value Analysis (BVA)**.

1. **Boundary Value Analysis:** Applied to ensure the extreme limits of the valid range (1 and 3999) work correctly, and that values immediately outside those boundaries (0 and 4000) are handled safely.

2. **Equivalence Partitioning:** Applied to categorize inputs into "Valid" and "Invalid" buckets. For valid integers, I ensured I tested numbers requiring additive notation and subtractive notation. For invalid inputs, I included negative and incorrect data types.

3. **Error Guessing:** Specifically, for the `romanToInteger` function, I anticipated common invalid Roman numeral formatting.

Manual Test Cases: `integerToRoman(int)`:

- **TC-M-I2R-01 (Lower Boundary Valid):** `integerToRoman(1) => "I"`
- **TC-M-I2R-02 (Upper Boundary Valid):** `integerToRoman(3999) => "MMMCMXCIX"`
- **TC-M-I2R-03 (Equivalence Partition - Subtractive Logic):** `integerToRoman(494) => "CDXCIV"`
- **TC-M-I2R-04 (Equivalence Partition - Additive Logic):** `integerToRoman(28) => "XXVIII"`
- **TC-M-I2R-05 (Lower Boundary Invalid):** `integerToRoman(0)` => *Error / Exception*

- **TC-M-I2R-06 (Upper Boundary Invalid):** `integerToRoman(4000)` => *Error / Exception*
- **TC-M-I2R-07 (Invalid Partition - Negative):** `integerToRoman(-10)` => *Error / Exception*
- **TC-M-I2R-08 (Invalid Partition - Float):** `integerToRoman(15.5)` => *Error / Exception*

Manual Test Cases: romanToInteger(String)

- **TC-M-R2I-01 (Lower Boundary Valid):** romanToInteger("I") => 1
- **TC-M-R2I-02 (Upper Boundary Valid):** romanToInteger("MMMCMXCIX") => 3999
- **TC-M-R2I-03 (Equivalence Partition - Complex String):** romanToInteger("MCMXCIV") => 1994
- **TC-M-R2I-04 (Invalid Partition - Bad Characters):** romanToInteger("MMXQ") => *Error / Exception*
- **TC-M-R2I-05 (Invalid Grammar - Illegal Repetition):** romanToInteger("IIII") => *Error / Exception*
- **TC-M-R2I-06 (Invalid Grammar - Illegal Subtraction):** romanToInteger("IC") => *Error / Exception*
- **TC-M-R2I-07 (Invalid Grammar - Non-repeatable chars):** romanToInteger("VV") => *Error / Exception*
- **TC-M-R2I-08 (Invalid Partition - Empty String):** romanToInteger("") => *Error / Exception*

3. Test Cases with AI

The promps we used for this part are:

Prompt 1 (System Rules):

"Act as a Senior QA Engineer. We need to test two functions: `integerToRoman(num)` and `romanToInteger(str)`. The constraints are strict: valid numbers are integers from 1 to 3999 only. No 0, no negatives, no decimals. There is only one exact correct string per number. Invalid Roman grammar (like 'IIII' or 'IL') must fail."

Prompt 2 (Generate I2R):

"Generate a comprehensive suite of functional test cases for `integerToRoman(num)`. Group them by categories (Boundaries, Valid Equivalence, Invalid Inputs). Use the format: TC-AI-I2R-XX: integerToRoman(input) => 'Expected Output'."

Prompt 3 (Generate R2I):

"Now do the exact same for `romanToInteger(str)`. Ensure you heavily test invalid Roman numeral structures based on standard grammatical rules, as well as boundary and valid inputs."

AI-Assisted Test Cases: `integerToRoman(int)`

Boundary Value Testing:

- **TC-AI-I2R-01:** `integerToRoman(1)` => `"I"` (Absolute Minimum)
- **TC-AI-I2R-02:** `integerToRoman(3999)` => `"MMMCMXCIX"` (Absolute Maximum)

Equivalence Partitioning (Valid Data):

- **TC-AI-I2R-03:** `integerToRoman(10)` => `"X"` (Single character representation)
- **TC-AI-I2R-04:** `integerToRoman(58)` => `"LVIII"` (Complex additive combination)
- **TC-AI-I2R-05:** `integerToRoman(1994)` => `"MCMXCIV"` (Complex subtractive combination)

Negative Testing (Invalid Data):

- **TC-AI-I2R-06:** `integerToRoman(0)` => *Error* (Zero not allowed)
- **TC-AI-I2R-07:** `integerToRoman(4000)` => *Error* (Out of upper bounds)
- **TC-AI-I2R-08:** `integerToRoman(-50)` => *Error* (Negative numbers)
- **TC-AI-I2R-09:** `integerToRoman(null)` => *Error* (Null handling)

AI-Assisted Test Cases: `romanToInteger(String)`

Boundary Value Testing:

- **TC-AI-R2I-01:** `romanToInteger("I")` => `1`
- **TC-AI-R2I-02:** `romanToInteger("MMMCMXCIX")` => `3999`

Equivalence Partitioning (Valid Data):

- **TC-AI-R2I-03:** `romanToInteger("IX")` => `9`
- **TC-AI-R2I-04:** `romanToInteger("LVIII")` => `58`
- **TC-AI-R2I-05:** `romanToInteger("MCMXCIV")` => `1994`

Negative Testing (Invalid Roman Grammar):

- **TC-AI-R2I-06:** `romanToInteger("IIII")` => *Error* (More than 3 sequential repeats of I, X, C, M)
- **TC-AI-R2I-07:** `romanToInteger("VV")` => *Error* (Repeated V, L, D which cannot be repeated)
- **TC-AI-R2I-08:** `romanToInteger("IL")` => *Error* (Invalid subtractive notation; I can only precede V and X)
- **TC-AI-R2I-09:** `romanToInteger("CMC")` => *Error* (Invalid sequence)
- **TC-AI-R2I-10:** `romanToInteger("MCMXCIVI")` => *Error* (Invalid trailing characters)

Negative Testing (Data Types & Characters):

- **TC-AI-R2I-11:** `romanToInteger("123")` => *Error* (Numeric string)
- **TC-AI-R2I-12:** `romanToInteger("MC M")` => *Error* (Contains whitespace)
- **TC-AI-R2I-13:** `romanToInteger(null)` => *Error* (Null handling)
- **TC-AI-R2I-14:** `romanToInteger("mcmxciv")` => *Error / 1994* (Depending on strict case sensitivity rules; generally should be tested for explicit handling)

4. Comparison and Analysis

After creating cases manually and using AI, several key differences and observations emerged:

1. Structure and Categorization

- **Manual Approach:** My manual approach was organic. I thought of the rules (like boundaries and negative numbers) and listed the cases as they logically flowed in my mind.
- **AI Approach:** The AI naturally grouped the test cases into strict, formal categories (Boundary Testing, Equivalence Partitioning, Negative Testing). This makes the AI-generated test suite much easier to read and present to stakeholders.

2. Exhaustiveness of Negative Testing

- **Manual Approach:** I successfully identified the primary logical errors (like "IC" instead of XCIX, or "IIII").
- **AI Approach:** The AI was much more thorough with edge cases regarding formatting and data sanitization. It included tests for whitespace ("MC M"), specific invalid repetitions of non-repeatable numerals ("VV"), trailing garbage characters ("MCMXCIVI"), and programmatic edge cases like null inputs.

3. Rule Memorization vs. Discovery

- **Manual Approach:** Designing tests manually required me to actively lookup and recall specific Roman numeral rules (e.g., remembering that I can only be subtracted from V and X).
- **AI Approach:** Because the AI was pre-trained on these historical mathematical rules, it immediately generated highly specific grammatical violations (like TC-AI-R2I-08 testing "IL") without needing those specific rules detailed in the prompt.

5. Conclusion

The manual method ensured a strong foundational understanding of the exact requirements provided for the task. However, the AI-assisted method was superior in rapidly building out a robust, structurally organized test suite that accounted for programmatic edge cases (whitespace, nulls, strict grammar) that a human might

accidentally overlook in an initial draft. Using them together—human strategy setting the constraints, AI generating the exhaustive permutations—yields the best QA coverage.

Actualización del Informe: Fase de Ejecución y Refactorización

1. Resumen de la Ejecución Automatizada

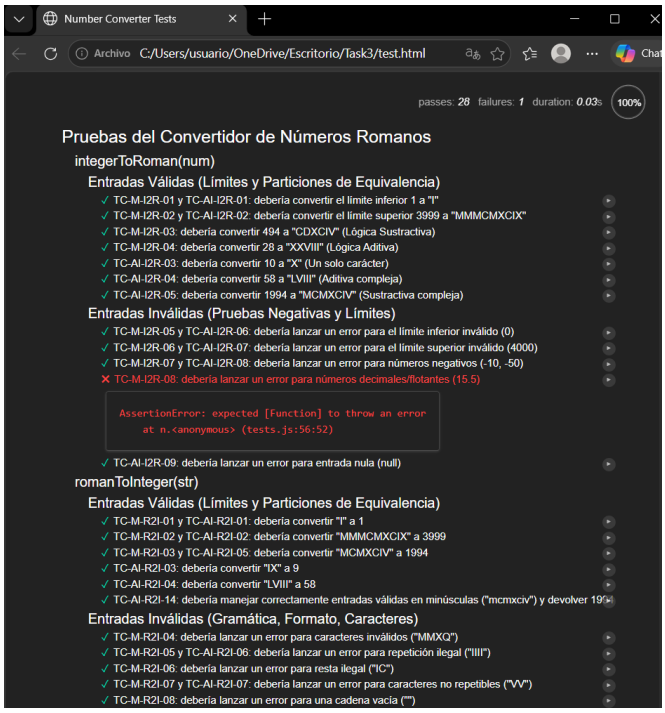
Tras implementar la suite de pruebas en **Mocha/Chai**, se realizó una ejecución inicial sobre la lógica original. Los resultados fueron los siguientes:

- **Pruebas totales:** 29
- **Pruebas superadas:** 28
- **Pruebas fallidas:** 1 (TC-M-I2R-08)

2. Identificación del "Bug" (Error de Lógica)

El sistema falló en la prueba de **Partición de Equivalencia Inválida** para números decimales.

- **Entrada:** 15.5
- **Comportamiento esperado:** Lanzar un error (Error: The number must be an integer).
- **Comportamiento observado:** El sistema devolvía "XV", ignorando la parte decimal. Esto se debía a que la función integerToRoman solo validaba límites numéricos (1-3999) pero no el tipo de dato (Integer), permitiendo que el bucle while procesara solo la parte entera.



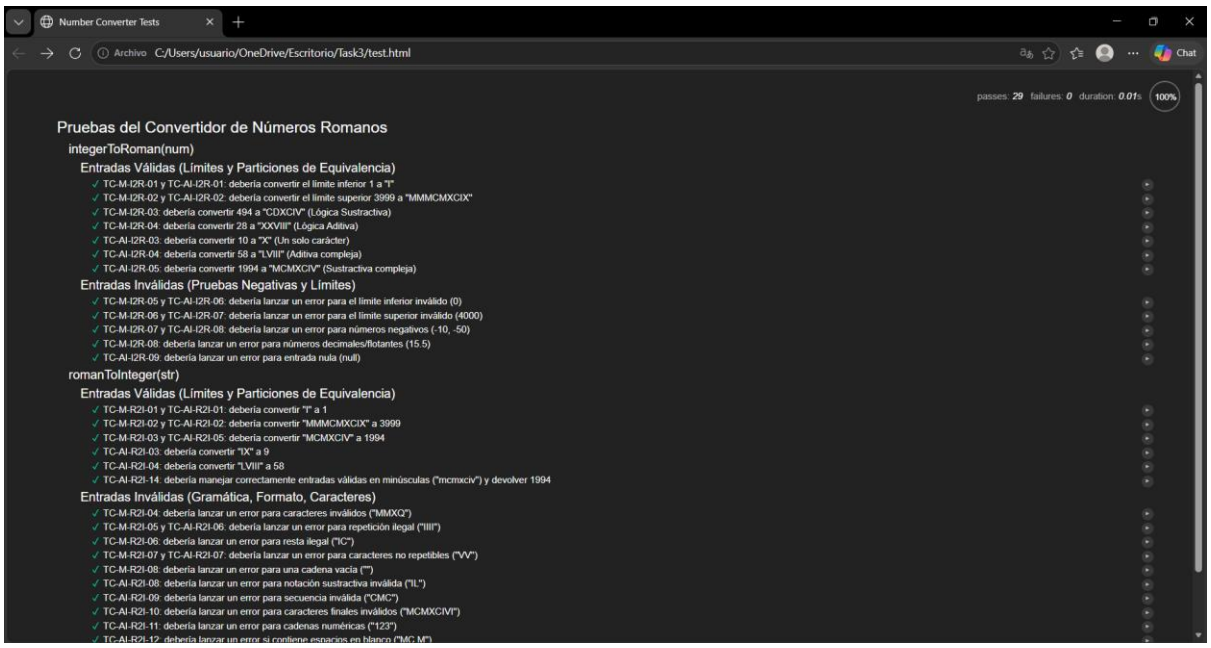
3. Acción Correctiva (Refactorización)

Para garantizar la integridad de los requisitos de calidad, se aplicaron dos cambios técnicos:

1. **Validación de Tipo:** Se añadió la instrucción Number.isInteger(num) en la función principal para bloquear cualquier entrada que no sea un entero puro.

2. **Sincronización de Interfaz:** Se ajustó handleConversion para usar Number() en lugar de parseInt(), asegurando que los decimales lleguen a la lógica de validación en lugar de ser truncados preventivamente.

Código corregido: if (!Number.isInteger(num)) { throw new Error('The number must be an integer.'); }



4. Verificación Final

Tras la corrección, se procedió a una re-ejecución completa. El sistema alcanzó un 100% de éxito (29/29 pruebas en verde).